

Parallel Algorithms

J. Daniel García

Universidad Carlos III de Madrid



Creative Commons License

© This work is distributed under the **Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)** license.

You are free to:

Share – copy and redistribute the material in any medium or format.

Adapt – remix, transform, and build upon the material.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

- ① **Attribution** – You must give **appropriate credit**, provide a link to the license, and **indicate if changes were made**. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- ⑤ **NonCommercial** – You may not use the material for **commercial purposes**.
- ③ **ShareAlike** – If you **remix, transform, or build upon** the material, you must distribute the modified material under the same license.

No additional restrictions – You may not apply legal terms or **technological measures** that legally restrict others from doing anything the license permits.

Complete text of the license available at

<https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode.en>

- 1 The C++ programming language
- 2 Algorithms in the STL
- 3 Parallel execution of algorithms
- 4 Parallel numeric algorithms

What is C++?

- **“C++ is a lightweight abstractions programming language”** (B. Stroustrup).

What is C++?

- **“C++ is a lightweight abstractions programming language”** (B. Stroustrup).
- Continuously evolving since 1983, with a new standard every 3 years since 2011.
 - Many compatibility elements.
 - Some of them tracing back to 1972.
 - Evolving a language is hard, and breaking compatibility is even harder.
 - Billions of lines of code, and millions of developers.

What is C++?

- **“C++ is a lightweight abstractions programming language”** (B. Stroustrup).
- Continuously evolving since 1983, with a new standard every 3 years since 2011.
 - Many compatibility elements.
 - Some of them tracing back to 1972.
 - Evolving a language is hard, and breaking compatibility is even harder.
 - Billions of lines of code, and millions of developers.
- **“When someone says “I want a programming language in which I need only say what I wish done,” give him a lollipop.”** (A. Perlis).

A brief history of C++

- 1979: **C with classes**.
- 1983: **C++ 1.0**.
- 1989: **C++ 2.0**.
- 1998: **C++98** → **ISO/IEC 14882:1998**.
- 2003: **C++03** = C++98 with bug fixes.
- 2011: **C++11** → **ISO/IEC 14882:2011**.
- 2014: **C++14** = C++11 with bug fixes and small improvements.
- 2017: **C++17** → **ISO/IEC 14882:2017**.
- 2020: **C++20** → **ISO/IEC 14882:2020**.
- 2023: **C++23** (published 2024) → **ISO/IEC 14882:2024**.
- 2026: **C++26** → **ISO/IEC 14882:2026 (expected)**.

First X3J16 meeting

Somerset, NJ, USA
(1990)



Completed C++11

Madrid, Spain
(2011)





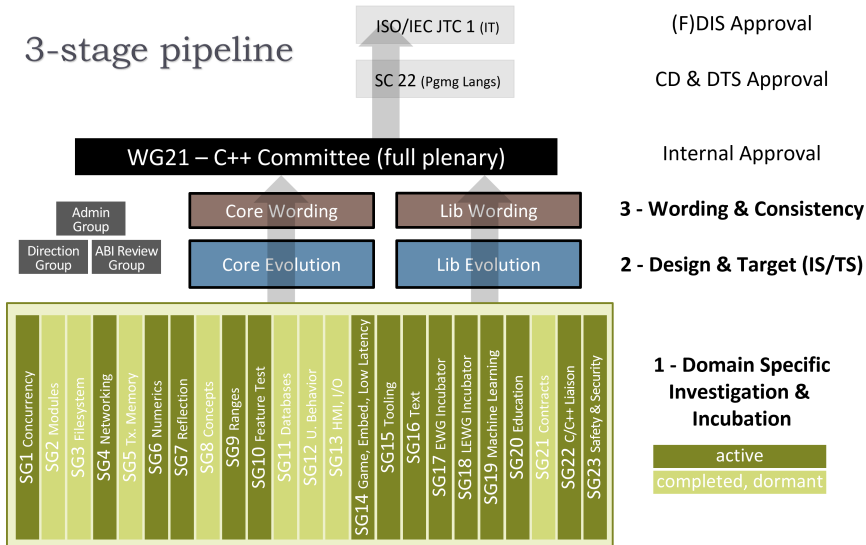




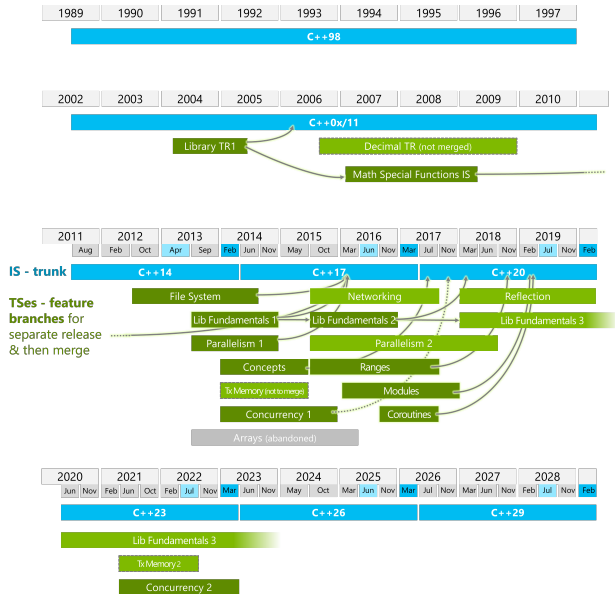




ISO C++ committee structure



ISO C++ timeline



- Keep **backwards compatibility**.

Design principles for C++

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.

Design principles for C++

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.
- Ease the design of **systems and libraries** (instead of concrete domains).

Design principles for C++

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.
- Ease the design of **systems and libraries** (instead of concrete domains).
- Improve **type safety**.

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.
- Ease the design of **systems and libraries** (instead of concrete domains).
- Improve **type safety**.
- Improve **performance** and **interaction with hardware**.

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.
- Ease the design of **systems and libraries** (instead of concrete domains).
- Improve **type safety**.
- Improve **performance** and **interaction with hardware**.
- **Zero-overhead** principle.

- Keep **backwards compatibility**.
- Better to **extend the library** than the language.
- Ease the design of **systems and libraries** (instead of concrete domains).
- Improve **type safety**.
- Improve **performance** and **interaction with hardware**.
- **Zero-overhead** principle.
- Make C++ easier to **teach and learn**.

2 Algorithms in the STL

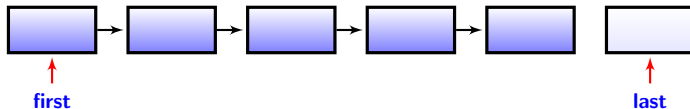
- Containers and iterators
- Generic algorithms

- **Container:** Object capable of storing other objects.
 - Manage acquisition and release of memory for the stored elements.
 - Take care of insertion, removal, copy, ...

- **Container:** Object capable of storing other objects.
 - Manage acquisition and release of memory for the stored elements.
 - Take care of insertion, removal, copy, ...
- **Sequence container:** Logical linear organization.
 - `std::vector<T>`: Variable number of elements contiguously stored in dynamic memory.
 - `std::list<T>`: Doubly linked list.
 - `std::forward_list<T>`: Singly linked list.
 - `std::deque<T>`: Double ended queue.
 - `std::array<T,N>`: Fixed number of elements stored contiguously, without dynamic memory.

- **Iterator**: Object referring to location in a container that allows to manipulate the value stored in that location.
 - It is a generalization of the pointer concept.

- **Iterator**: Object referring to location in a container that allows to manipulate the value stored in that location.
 - It is a generalization of the pointer concept.
- A **range** can be represented as a pair of iterators:
 - **First**: Iterator to the first element in the sequence.
 - **Last**: Iterator to one past the last element in the sequence.
 - **Range**: **Semi-open** interval **[first,last)**.



- **Type aliases** for iterators in all **containers**.
 - **iterator**: Iterator to traverse the container.
 - They **allow** modifying pointed elements.
 - **const_iterator**: Iterator to traverse container.
 - They **do not allow** modifying pointed elements.

- **Type aliases** for iterators in all **containers**.
 - **iterator**: Iterator to traverse the container.
 - They **allow** modifying pointed elements.
 - **const_iterator**: Iterator to traverse container.
 - They **do not allow** modifying pointed elements.
- **Iterator values** for **ends** in a container.
 - **c.begin()**: Iterator to the start of **c**.
 - **c.end()**: Iterator to the end of **c**.
 - **c.cbegin()** / **c.cend()**: Constant iterators.

- **Input iterator**: Allows advancing and reading only once (stream reading).
- **Output iterator**: Allows advancing and writing only once (stream writing).
- **Forward iterator**: Allows advancing and reading/writing multiple times (simply linked lists)
- **Bidirectional iterator**: Additionally allows to move backwards (doubly linked lists).
- **Random access iterator**: Allows access to a location with an offset.

Iterator categories hierarchy

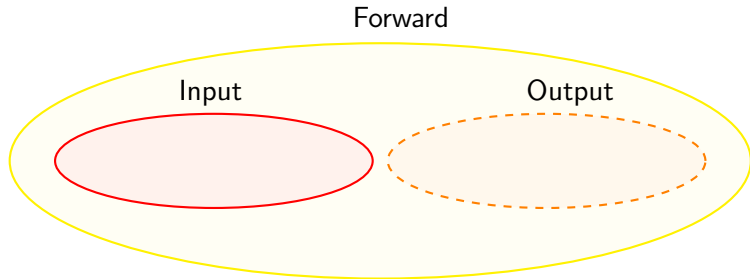
Input



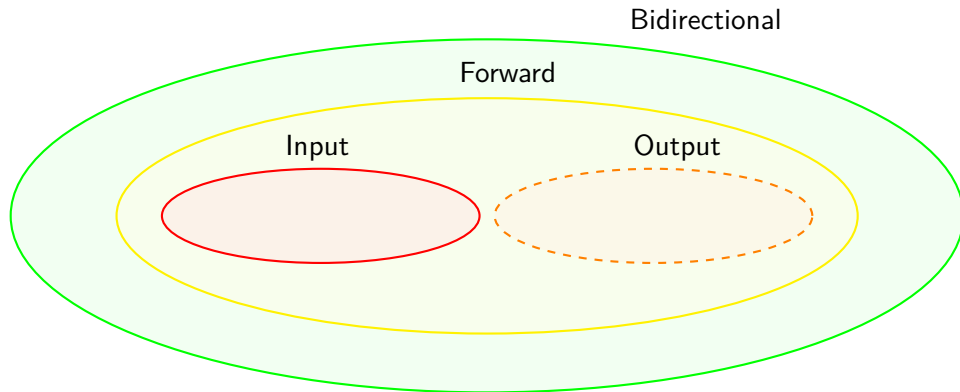
Iterator categories hierarchy



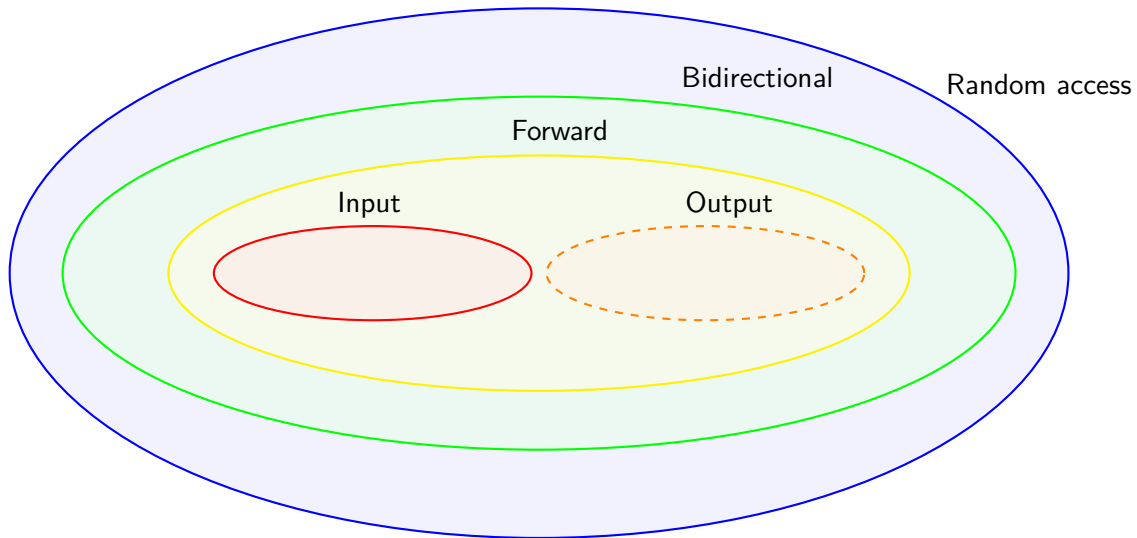
Iterator categories hierarchy



Iterator categories hierarchy



Iterator categories hierarchy



Using forward iterators

- Can be used on all sequence containers.

```
double add(std::forward_list<double> const & lst) {  
    double res = 0.0;  
    auto first = lst.begin();  
    auto last = lst.end();  
    while (first != last) {  
        res += *first++;  
    }  
    return res;  
}
```


Using forward iterators

- Can be used on all sequence containers.

```
double add(std::forward_list<double> const & lst) {  
    double res = 0.0;  
    auto first = lst.begin();  
    auto last = lst.end();  
    while (first != last) {  
        res += *first++;  
    }  
    return res;  
}
```

- **Does not allow:**

- Going backwards.
- Accessing a location with an offset.
- Advancing or going backwards several steps in one operation.

- Can be used on containers that allow traversal in both directions.

```
void invert(std::list<double> & lst) {  
    auto first = lst.begin();  
    auto last = lst.end();  
    auto len = std::distance(first,last) - 1; // or lst.size()-1  
    while (len>0) {  
        std::swap(*first++, *--last);  
        len -= 2;  
    }  
}
```

- Can be used on containers that allow traversal in both directions.

```
void invert(std::list<double> & lst) {  
    auto first = lst.begin();  
    auto last = lst.end();  
    auto len = std::distance(first, last) - 1; // or lst.size()-1  
    while (len > 0) {  
        std::swap(*first++, *--last);  
        len -= 2;  
    }  
}
```

- **Does not allow:**

- Accessing a location with an offset.
- Advancing or going backwards several steps in one operation.

Using random access iterators

- Can be used on containers that allow random access.

```
std::vector<double>::const_iterator find_sorted(std::vector<double> const & vec, double value) {  
    auto first = vec.begin();  
    auto last = vec.end();  
    while (first < last) {  
        auto dist = std::distance(first, last);  
        auto mid = first + dist / 2;  
        if (*mid == value) { return mid; }  
        if (*mid < value) { first = mid + 1; }  
        else { last = mid; }  
    }  
    return vec.end();  
}
```

2 Algorithms in the STL

- Containers and iterators
- Generic algorithms

- Layer separation
 - **Containers**: Data structures storing information.
 - **Iterators**: Locations in containers used to access and manipulate data.
 - **Algorithms**: Operations defined in terms of iterators.

- Layer separation
 - **Containers**: Data structures storing information.
 - **Iterators**: Locations in containers used to access and manipulate data.
 - **Algorithms**: Operations defined in terms of iterators.

- **Advantages**:
 - **Separation of concerns**: Each layer has well defined responsibilities.
 - **Extensibility**: Each layer can be extended independently.
 - **Specialization**: Different versions of the same algorithm for different iterators.
 - **Optimization**: Compile-time decisions to improve generated code.
 - **Parallelism**: Single interface for sequential and parallel versions.

Algorithm categories

- Great variety of algorithms.
 - Full list: <https://en.cppreference.com/w/cpp/algorithm.html>
 - More than 100 algorithms.
 - More algorithms, beyond sequence algorithms.

- Great variety of algorithms.
 - Full list: <https://en.cppreference.com/w/cpp/algorithm.html>
 - More than 100 algorithms.
 - More algorithms, beyond sequence algorithms.
- **Categories:**
 - Non-modifying sequence operations.
 - Modifying sequence operations.
 - Sorting operations.
 - Numeric operations.

Simple find

```
template< class InputIt, class T >  
InputIt find(InputIt first, InputIt last, T const & value );
```

- Find the first element in **[first,last)** equal to **value**.
- Returns **last** if no occurrence is found.

```
std::list lst{-1, 3, 2, 4, -5, 7};  
auto it1 = std::find(lst.begin(), lst.end(), 2); // it1 points to third element  
auto it2 = std::find(lst.begin(), lst.end(), 1); // it2 == lst.end()
```

Higher order functions

- Many algorithms need to receive another function as an argument:
 - `std::find_if(first, last, criterion)`.
 - `std::find_if_not(first, last, criterion)`.

- Many algorithms need to receive another function as an argument:
 - `std::find_if(first, last, criterion)`.
 - `std::find_if_not(first, last, criterion)`.

- **Lambda expression:** Defines an unnamed function in a local context.

```
std::list lst{-1, 3, 2, 4, -5, 7};  
std::find_if(lst.begin(), lst.end(), [](int x) { return x>0 and x%2==0; });  
std::find_if_not(lst.begin(), lst.end(), [](int x) { return x<0; });
```

- **Capture:** Introduces lambda and allows information passing.

// Lambda without capture

```
std::find_if(lst.begin(), lst.end(), [](int x) { return x>0; });
```

- **Capture:** Introduces lambda and allows information passing.

// Lambda without capture

```
std::find_if(lst.begin(), lst.end(), [](int x) { return x>0; });
```

// Lambda with by-value capture. Access to max for reading.

```
int max = 3;
```

```
std::find_if(lst.begin(), lst.end(), [=](int x) { return x>max; });
```

- **Capture:** Introduces lambda and allows information passing.

// Lambda without capture

```
std::find_if(lst.begin(), lst.end(), [](int x) { return x>0; });
```

// Lambda with by-value capture. Access to max for reading.

```
int max = 3;
```

```
std::find_if(lst.begin(), lst.end(), [=](int x) { return x>max; });
```

// Lambda with by-reference capture. Access for modification.

```
int n = 0;
```

```
std::find_if(lst.begin(), lst.end(), [&](int x) { n++; return x>max; });
```

- **Capture:** Introduces lambda and allows information passing.

// Lambda without capture

```
std::find_if(lst.begin(), lst.end(), [](int x) { return x>0; });
```

// Lambda with by-value capture. Access to max for reading.

```
int max = 3;
```

```
std::find_if(lst.begin(), lst.end(), [=](int x) { return x>max; });
```

// Lambda with by-reference capture. Access for modification.

```
int n = 0;
```

```
std::find_if(lst.begin(), lst.end(), [&](int x) { n++; return x>max; });
```

- **Return type:** Automatically deduced from return expression.

- Can be explicitly specified.

```
std::find(lst.begin(), lst.end(), [](int x) { return x>0; });
```

```
std::find(lst.begin(), lst.end(), ([](int x) -> bool { return x>0; }));
```


Counting elements

```
template< class InputIt, class T >
typename std::iterator_traits<InputIt>::difference_type
count( InputIt first, InputIt last, const T& value );
```

```
template< class InputIt, class UnaryPred >
typename std::iterator_traits<InputIt>::difference_type
count_if( InputIt first, InputIt last, UnaryPred p );
```

- Counts the number of elements that are equal to a value or that satisfy a predicate.
 - A unary predicate takes a single parameter and returns a boolean.

```
std::vector<std::string> leng{"C++", "Go", "C", "COBOL", "Java"};
auto n = std::count_if(leng.begin(), leng.end(),
    [](std::string const & name) { return name.size()>0 and name[0]=='C'; }
);
```

3 Parallel execution of algorithms

- Parallel algorithms
 - Using parallel algorithms
 - Batch algorithms
 - Search algorithms
 - Modifying sequence algorithms
 - Other algorithms

- Many algorithms accept an **execution policy**:

```
std::vector<double> v = get_values();  
auto n1 = std::count_if(v.begin(), v.end(), [](double x) { return x>0; });  
auto n2 = std::count_if(std::execution::seq, v.begin(), v.end(), [](double x) { return x>0; });  
auto n3 = std::count_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>0; });
```

- Many algorithms accept an **execution policy**:

```
std::vector<double> v = get_values();  
auto n1 = std::count_if(v.begin(), v.end(), [](double x) { return x>0; });  
auto n2 = std::count_if(std::execution::seq, v.begin(), v.end(), [](double x) { return x>0; });  
auto n3 = std::count_if(std::execution::par, v.begin(), v.end(), [](double x) { return x>0; });
```

- Counting values that satisfy a predicate with **count_if()**:

```
#include <algorithm>  
#include <execution>  
#include <print>  
#include <vector>  
#include "image.hpp"  
  
int main() {  
    std::vector<image> v = load_images("img.dat");  
    auto n = std::count_if(std::execution::par, v.begin(), v.end(),  
        [](image const & im) { return not im.is_valid(); });  
    std::println("Invalid images = {}", n);  
}
```

- **std::execution::seq:**
 - Execution is sequenced as if on a single thread (no parallelism or vectorization assumed).
- **std::execution::par:**
 - May execute iterations in parallel, but each iteration is sequential (no vectorization assumed).
- **std::execution::par_unseq:**
 - May execute iterations in parallel and unsequenced; operations may be vectorized.
- **std::execution::unseq:**
 - Executes unsequenced within a single thread context; operations may be vectorized.

- If an exception is thrown by element access, predicate, or operation during an algorithm invoked with a `std::execution` policy, `std::terminate()` is called.

Parallel algorithms and exceptions

- If an exception is thrown by element access, predicate, or operation during an algorithm invoked with a `std::execution` policy, `std::terminate()` is called.
- This includes `std::execution::seq`, even though it is not parallel.
 - Does not apply to non-`std::execution`-based algorithms.

Parallel algorithms and exceptions

- If an exception is thrown by element access, predicate, or operation during an algorithm invoked with a `std::execution` policy, `std::terminate()` is called.
- This includes `std::execution::seq`, even though it is not parallel.
 - Does not apply to non-`std::execution`-based algorithms.
- An algorithm may still throw `std::bad_alloc` if memory allocation fails, even when using a parallel execution policy.

Parallel algorithms and exceptions

- If an exception is thrown by element access, predicate, or operation during an algorithm invoked with a `std::execution` policy, `std::terminate()` is called.
- This includes `std::execution::seq`, even though it is not parallel.
 - Does not apply to non-`std::execution`-based algorithms.
- An algorithm may still throw `std::bad_alloc` if memory allocation fails, even when using a parallel execution policy.
- **Rationale:** Parallel algorithms may execute work on multiple threads or in vectorized/unsequenced contexts, and the execution-policy overloads do not support propagating exceptions from user code to the caller.
 - Calling `std::terminate()` keeps the semantics simple for implementations and preserves optimization freedom for parallel and vectorized execution.

3 Parallel execution of algorithms

- Parallel algorithms
- Using parallel algorithms
- Batch algorithms
- Search algorithms
- Modifying sequence algorithms
- Other algorithms

Applying an operation

```
template <class ExecutionPolicy, class ForwardIt, class UnaryFunc>  
void for_each(ExecutionPolicy&& policy, ForwardIt first, ForwardIt last, UnaryFunc f);
```

- Applies unary function **f()** to each element in range **[first,last)**.

Applying an operation

```
template <class ExecutionPolicy, class ForwardIt, class UnaryFunc>
void for_each(ExecutionPolicy&& policy, ForwardIt first, ForwardIt last, UnaryFunc f);
```

- Applies unary function **f()** to each element in range **[first,last)**.

```
#include <algorithm>
#include <execution>
#include <vector>
#include "image.hpp"

int main() {
    std::vector<image> v = load_images("img.dat");
    std::for_each(std::execution::par, v.begin(), v.end(),
        [](image & im) { im.to_gray(); });
    store_images(v, "img2.dat");
}
```

Transformations

```
template <class ExecutionPolicy, class ForwardIt1, class ForwardIt2, class UnaryOp>  
ForwardIt2 transform(ExecutionPolicy&& policy, ForwardIt1 first1, ForwardIt1 last1,  
                    ForwardIt2 d_first, UnaryOp unary_op);
```

- A **transformation** (also known as **map** operation) transforms elements in a sequence and produces a new sequence.

```
template <class ExecutionPolicy, class ForwardIt1, class ForwardIt2, class UnaryOp>
ForwardIt2 transform(ExecutionPolicy&& policy, ForwardIt1 first1, ForwardIt1 last1,
                    ForwardIt2 d_first, UnaryOp unary_op);
```

- A **transformation** (also known as **map** operation) transforms elements in a sequence and produces a new sequence.
- For each element **x** in the range **[first1, last1)** writes the value **f(x)** starting at **[d_first, ...)**.

```
std::vector<double> square(std::vector<double> const & v) {
    std::vector<double> r(v.size());
    std::transform(std::execution::par, v.begin(), v.end(), r.begin(),
        [](double x) { return x*x; });
    return r;
}
```

Binary transformations

```
template<class ExecutionPolicy, class ForwardIt1, class ForwardIt2, class ForwardIt3, class BinaryOp>  
ForwardIt3 transform(ExecutionPolicy&& policy, ForwardIt1 first1, ForwardIt1 last1, ForwardIt2 first2,  
                    ForwardIt3 d_first, BinaryOp binary_op);
```

```
template<class ExecutionPolicy, class ForwardIt1, class ForwardIt2, class ForwardIt3, class BinaryOp>
ForwardIt3 transform(ExecutionPolicy&& policy, ForwardIt1 first1, ForwardIt1 last1, ForwardIt2 first2,
                    ForwardIt3 d_first, BinaryOp binary_op);
```

- For each element **x** in the range **[first1, last1)** and **y** in the range **[first2,...)** writes the value **f(x,y)** starting from **[d_first,...)**.

```
std::vector<double> add(std::vector<double> const & v, std::vector<double> const & w) {
    std::vector<double> r(v.size());
    std::transform(std::execution::par, v.begin(), v.end(), w.begin(), r.begin(),
        [](double x, double y) { return x+y; });
    return r;
}
```


Reductions

```
template <class ExecutionPolicy, class ForwardIt>  
    typename std::iterator_traits<ForwardIt>::value_type  
reduce(ExecutionPolicy&& policy, ForwardIt first, ForwardIt last);
```

```
template <class ExecutionPolicy, class ForwardIt>
    typename std::iterator_traits<ForwardIt>::value_type
    reduce(ExecutionPolicy&& policy, ForwardIt first, ForwardIt last);
```

- Combines all the elements in range **[first,last)** in a single value, using a combination operation and an initial value.
 - Combination operation is addition.
 - The initial value is the default value (usually **0**).

```
void print_addition(std::vector<double> const & v) {
    auto r = std::reduce(std::execution::par, v.begin(), v.end());
    std::println("Sum = {}", r);
}
```

Other forms of reduction

- Additionally the following can be provided:

- Initial value.
- Initial value and reduction operation.

```
double sum_squares(std::vector<double> const & v) {  
    return std::transform_reduce(v.begin(), v.end(), 0.0,  
        [](double x, double y) { return x+y; }, // Reduce stage  
        [](double y) { return y * y; } // Map stage  
    );  
}
```

- A **map-reduce** operation combines a **transformation** operation (**map** operation) with a **reduction** of its partial results (**reduce** operation).

```
double norm(std::vector<double> const & v) {  
    auto sum = std::transform_reduce(std::execution::par, v.begin(), v.end(),  
        0.0, // initial value  
        [](double x, double y) { return x+y; }, // Reduce stage  
        [](double x) { return x*x; } // Map stage  
    );  
    return std::sqrt(sum);  
}
```

Word frequency in text

- Starting from a vector of words: `std::vector<std::string>`.
- Generate a dictionary of word-frequency: `std::map<std::string, std::size_t>`.

```
std::map<std::string, std::size_t> frequencies(std::vector<std::string> const & v) {  
    using dictionary = std::map<std::string, std::size_t>;  
    return std::transform_reduce(std::execution::par, v.begin(), v.end(),  
        dictionary{}, // empty dictionary  
        // Combining dictionaries  
        [](dictionary d1, dictionary const & d2) -> dictionary {  
            for (auto const & [key, value] : d2) { d1[key] += value; }  
            return d1;  
        },  
        // Generate a dictionary with a word and frequency 1  
        [](std::string const & word) { return dictionary{{{to_lowercase(word), 1}}}; }  
    );  
}
```

3 Parallel execution of algorithms

- Parallel algorithms
- Using parallel algorithms
- **Batch algorithms**
- Search algorithms
- Modifying sequence algorithms
- Other algorithms

Batch algorithms

- **Batch algorithms** apply a unary operation to each element in a sequence.

Batch algorithms

- **Batch algorithms** apply a unary operation to each element in a sequence.
- Sequence defined by a pair of iterators **[f, l)**.
`UnaryOp for_each(f, l, unary_op);`
- Sequence defined by an iterator **f** and a count **n**.
`Iterator for_each_n(f, n, unary_op);`

- **Batch algorithms** apply a unary operation to each element in a sequence.
- Sequence defined by a pair of iterators **[f, l]**.
`UnaryOp for_each(f, l, unary_op);`
- Sequence defined by an iterator **f** and a count **n**.
`Iterator for_each_n(f, n, unary_op);`
- **Differences** in **execution-based** algorithms:
 - **Execution policy** overloads available for both interfaces.
 - Forward iterators instead of Input iterators.
 - **for_each** return type differs: execution-policy returns **void**.
 - **for_each_n** return type differs: execution-policy returns **ForwardIt**.
 - No copies of original elements are made.

- Iterator pair batch parallel interface:

```
template <class ExecutionPolicy, class ForwardIt, class UnaryFunc>  
void for_each(ExecutionPolicy&& policy,  
             ForwardIt first, ForwardIt last, UnaryFunc f);
```

- Iterator pair batch parallel interface:

```
template <class ExecutionPolicy, class ForwardIt, class UnaryFunc>
void for_each(ExecutionPolicy&& policy,
              ForwardIt first, ForwardIt last, UnaryFunc f);
```

- Iterator and size batch parallel interface:

```
template<class ExecutionPolicy,
         class ForwardIt, class Size, class UnaryFunc>
ForwardIt for_each_n(ExecutionPolicy&& policy,
                    ForwardIt first, Size n, UnaryFunc f);
```

Updating object positions

```
void update_positions(std::vector<particle>& vp, double dt) {  
    std::for_each(std::execution::par, vp.begin(), vp.end(), [dt](particle & p) {  
        p.position += p.velocity * dt;  
    });  
}
```

Applying batch operations

Updating object positions

```
void update_positions(std::vector<particle>& vp, double dt) {  
    std::for_each(std::execution::par, vp.begin(), vp.end(), [dt](particle & p) {  
        p.position += p.velocity * dt;  
    });  
}
```

Risks of batch algorithms

```
void print_max(std::vector<int> const & v) {  
    int max = std::numeric_limits<int>::min();  
    std::for_each(std::execution::par, v.begin(), v.end(), [&max](int x) {  
        if (x > max) {  
            max = x;  
        }  
    });  
    std::println("Max = {}", max);  
}
```

3 Parallel execution of algorithms

- Parallel algorithms
- Using parallel algorithms
- Batch algorithms
- **Search algorithms**
- Modifying sequence algorithms
- Other algorithms

Search algorithms with parallel versions

- `all_of()`, `any_of()`, `none_of()`.
- `find()`, `find_if()`, `find_if_not()`.
- `find_first_of()`, `find_end()`.
- `adjacent_find()`.
- `search()`, `search_n()`.
- `count()`, `count_if()`.
- `mismatch()`, `equal()`.

3 Parallel execution of algorithms

- Parallel algorithms
- Using parallel algorithms
- Batch algorithms
- Search algorithms
- **Modifying sequence algorithms**
- Other algorithms

Modifying sequence algorithms with execution-policy overloads

- `copy()`, `copy_if()`, `copy_n()`.
- `move()`.
- `swap_ranges()`.
- `replace()`, `replace_if()`, `replace_copy()`, `replace_copy_if()`.
- `fill()`, `fill_n()`, `generate()`, `generate_n()`.
- `remove()`, `remove_if()`, `remove_copy()`, `remove_copy_if()`.
- `unique()`, `unique_copy()`.
- `reverse()`, `reverse_copy()`.
- `rotate()`, `rotate_copy()`, `shift_left()`, `shift_right()`.

Modifying sequence algorithms with no execution-policy overloads

- `copy_backward()`.
- `move_backward()`.
- `shuffle()`, `random_shuffle()` (legacy; removed in C++17).
- `sample()`.

3 Parallel execution of algorithms

- Parallel algorithms
- Using parallel algorithms
- Batch algorithms
- Search algorithms
- Modifying sequence algorithms
- Other algorithms

Other algorithms with execution-policy overloads

- `is_partitioned()`, `partition()`, `partition_copy()`, `stable_partition()`.
- `is_sorted()`, `is_sorted_until()`.
- `sort()`, `stable_sort()`, `partial_sort()`, `partial_sort_copy()`, `nth_element()`.
- `merge()`, `inplace_merge()`.
- `is_heap()`, `is_heap_until()`.
- `includes()`, `set_difference()`, `set_intersection()`,
`set_symmetric_difference()`, `set_union()`.
- `min_element()`, `max_element()`, `minmax_element()`.
- `lexicographical_compare()`.

Other algorithms with no execution-policy overloads

- `partition_point()`.
- `lower_bound()`, `upper_bound()`, `equal_range()`.
- `binary_search()`.
- `push_heap()`, `pop_heap()`, `make_heap()`, `sort_heap()`.
- `is_permutation()`, `prev_permutation()`, `next_permutation()`.

4 Parallel numeric algorithms

- Differences with existing algorithms
- Reduction operations
- The map-reduce pattern
- Prefix sum operations

Changes in parallel numeric algorithms

- Parallel **numeric algorithms** support **execution policies**.
- Some name changes to reflect different **semantics**.
- Changes in iterator requirements: at least **forward iterators**.

Changes in parallel numeric algorithms

- Parallel **numeric algorithms** support **execution policies**.
- Some name changes to reflect different **semantics**.
- Changes in iterator requirements: at least **forward iterators**.

Old Interface	New Interface
<code>std::iota()</code>	—
<code>std::accumulate()</code>	<code>std::reduce()</code>
<code>std::inner_product()</code>	<code>std::transform_reduce()</code>
<code>std::adjacent_difference()</code>	<code>std::adjacent_difference()</code>
<code>std::partial_sum()</code>	<code>std::inclusive_scan()</code> <code>std::exclusive_scan()</code>
—	<code>std::transform_inclusive_scan()</code> <code>std::transform_exclusive_scan()</code>

4 Parallel numeric algorithms

- Differences with existing algorithms
- Reduction operations
- The map-reduce pattern
- Prefix sum operations

Accumulation versus reduction

- **std::accumulate**(**f**, **l**, **init**, **op**)
 - Accumulates values left to right sequentially.
 - Uses input iterators.
- **std::reduce**(**ex**, **f**, **l**, **init**, **op**)
 - Reduces values in any arbitrary order, possibly in parallel.
 - With an execution policy, uses forward iterators.
 - With an execution policy, binary operation **op** must be associative and commutative.

Accumulation versus reduction

- **std::accumulate(f, l, init, op)**
 - Accumulates values left to right sequentially.
 - Uses input iterators.
- **std::reduce(ex, f, l, init, op)**
 - Reduces values in any arbitrary order, possibly in parallel.
 - With an execution policy, uses forward iterators.
 - With an execution policy, binary operation **op** must be associative and commutative.
- **Consequences:**
 - **std::accumulate()** is not parallelizable.
 - **std::reduce()** can be parallelized, but execution-policy overloads require stronger iterator and operation requirements.
 - **std::reduce()** is not deterministic with non-associative or non-commutative operations.

Accumulation

```
auto add_seq(std::vector<int> const & vec) {  
    return std::accumulate(vec.begin(), vec.end(), 0);  
}
```

Accumulation

```
auto add_seq(std::vector<int> const & vec) {  
    return std::accumulate(vec.begin(), vec.end(), 0);  
}
```

Reduction

```
auto add_par(std::vector<int> const & vec) {  
    return std::reduce(std::execution::par, vec.begin(), vec.end(), 0);  
}
```

Adding squares of numbers

Accumulation

```
auto add_squares_seq(std::vector<int> const & vec) {  
    return std::accumulate(vec.begin(), vec.end(), 0, [](int acc, int x) {  
        return acc + x*x;  
    });  
}
```

Adding squares of numbers

Accumulation

```
auto add_squares_seq(std::vector<int> const & vec) {  
    return std::accumulate(vec.begin(), vec.end(), 0, [](int acc, int x) {  
        return acc + x*x;  
    });  
}
```

Reduction

```
auto add_squares_par(std::vector<int> const & vec) {  
    return std::reduce(std::execution::par, vec.begin(), vec.end(), 0, [](int acc, int x) {  
        return acc + x*x;  
    });  
}
```

- **Problem:** The binary operation is not associative, so the result may be non-deterministic.

4 Parallel numeric algorithms

- Differences with existing algorithms
- Reduction operations
- **The map-reduce pattern**
- Prefix sum operations

The map-reduce pattern

- Map-reduce is a common pattern for parallel algorithms.
- It consists of two steps:
 - **Map**: apply a function to each element of a sequence (or pairs of elements).
 - **Reduce**: combine the results of the mapping using an associative and commutative operation.

The map-reduce pattern

- Map-reduce is a common pattern for parallel algorithms.
- It consists of two steps:
 - **Map**: apply a function to each element of a sequence (or pairs of elements).
 - **Reduce**: combine the results of the mapping using an associative and commutative operation.
- **Variants**:
 - **Unary map-reduce**: map a unary function and reduce with a binary operation.
`T transform_reduce(f, l, init, binary_comb_op, unary_map_op);`
`T transform_reduce(ex, f, l, init, binary_comb_op, unary_map_op);`

The map-reduce pattern

- Map-reduce is a common pattern for parallel algorithms.
- It consists of two steps:
 - **Map**: apply a function to each element of a sequence (or pairs of elements).
 - **Reduce**: combine the results of the mapping using an associative and commutative operation.
- **Variants**:
 - **Unary map-reduce**: map a unary function and reduce with a binary operation.

```
T transform_reduce(f, l, init, binary_comb_op, unary_map_op);  
T transform_reduce(ex, f, l, init, binary_comb_op, unary_map_op);
```
 - **Binary map-reduce**: map a binary function on pairs of elements and reduce with a binary operation.

```
T transform_reduce(f1, l1, f2, init);  
T transform_reduce(ex, f1, l1, f2, init);  
T transform_reduce(f1, l1, f2, init, binary_comb_op, binary_map_op);  
T transform_reduce(ex, f1, l1, f2, init, binary_comb_op, binary_map_op);
```

Map-Reduce

```
auto add_squares_par(std::vector<int> const & vec) {  
    return std::transform_reduce(std::execution::par, vec.begin(), vec.end(), 0,  
        [](int a, int b) {  
            return a + b;  
        },  
        [](int x) {  
            return x*x;  
        });  
}
```

- **Solution:** The new combination operation is now associative and commutative, so the result is deterministic and can be parallelized; the map operation should also be pure.

4 Parallel numeric algorithms

- Differences with existing algorithms
- Reduction operations
- The map-reduce pattern
- Prefix sum operations

Prefix sum operations

- **Prefix sum** operations are a family of algorithms that compute the cumulative sums of a sequence of values.

Prefix sum operations

- **Prefix sum** operations are a family of algorithms that compute the cumulative sums of a sequence of values.
- **Definition:** Given a sequence of values $[x_1, x_2, \dots, x_n]$ and a binary operation **op**, the prefix sum of the sequence is defined as:
 - **Inclusive scan:**
 $[x_1, x_1 \text{ op } x_2, ((x_1 \text{ op } x_2) \text{ op } x_3), \dots, ((x_1 \text{ op } x_2) \text{ op } \dots \text{ op } x_n)]$
 - **Exclusive scan:**
 $[\text{init}, x_1, (x_1 \text{ op } x_2), ((x_1 \text{ op } x_2) \text{ op } x_3), \dots, ((x_1 \text{ op } x_2) \text{ op } \dots \text{ op } x_{n-1})]$

Prefix sum operations

- **Prefix sum** operations are a family of algorithms that compute the cumulative sums of a sequence of values.
- **Definition:** Given a sequence of values $[x_1, x_2, \dots, x_n]$ and a binary operation **op**, the prefix sum of the sequence is defined as:
 - **Inclusive scan:**
 $[x_1, x_1 \text{ op } x_2, ((x_1 \text{ op } x_2) \text{ op } x_3), \dots, ((x_1 \text{ op } x_2) \text{ op } \dots \text{ op } x_n)]$
 - **Exclusive scan:**
 $[\text{init}, x_1, (x_1 \text{ op } x_2), ((x_1 \text{ op } x_2) \text{ op } x_3), \dots, ((x_1 \text{ op } x_2) \text{ op } \dots \text{ op } x_{n-1})]$
- **Properties:**
 - The binary operation **op** must be associative.
 - The prefix sum can be computed in parallel using a tree-based algorithm.
 - The prefix sum preserves operand order, so the order of operations matters when **op** is non-commutative.

- **Inclusive scan** computes the cumulative sums including the current element.

- **Inclusive scan** computes the cumulative sums including the current element.

- **Generalized interface:**

```
l inclusive_scan(f, l, out_it, binary_combine_op, init);  
l inclusive_scan(ex, f, l, out_it, binary_combine_op, init);
```

- **Inclusive scan** computes the cumulative sums including the current element.

- **Generalized interface:**

```
l inclusive_scan(f, l, out_it, binary_combine_op, init);  
l inclusive_scan(ex, f, l, out_it, binary_combine_op, init);
```

- **No Initial value:**

```
l inclusive_scan(f, l, out_it, binary_combine_op);  
l inclusive_scan(ex, f, l, out_it, binary_combine_op);
```

- **Inclusive scan** computes the cumulative sums including the current element.

- **Generalized interface:**

```
l inclusive_scan(f, l, out_it, binary_combine_op, init);  
l inclusive_scan(ex, f, l, out_it, binary_combine_op, init);
```

- No **Initial value:**

```
l inclusive_scan(f, l, out_it, binary_combine_op);  
l inclusive_scan(ex, f, l, out_it, binary_combine_op);
```

- **Combine operation** defaulted to `std::plus<>`:

```
l inclusive_scan(f, l, out_it);  
l inclusive_scan(ex, f, l, out_it);
```

- **Exclusive scan** computes the cumulative sums excluding the current element.

- **Exclusive scan** computes the cumulative sums excluding the current element.

- **Generalized interface:**

```
l exclusive_scan(f, l, out_it, init, binary_combine_op);  
l exclusive_scan(ex, f, l, out_it, init, binary_combine_op);
```

- **Exclusive scan** computes the cumulative sums excluding the current element.

- **Generalized interface:**

```
| exclusive_scan(f, l, out_it, init, binary_combine_op);  
| exclusive_scan(ex, f, l, out_it, init, binary_combine_op);
```

- **Combine operation** defaulted to `std::plus<>`:

```
| exclusive_scan(f, l, out_it, init);  
| exclusive_scan(ex, f, l, out_it, init);
```

- **Exclusive scan** computes the cumulative sums excluding the current element.

- **Generalized interface:**

```
| exclusive_scan(f, l, out_it, init, binary_combine_op);  
| exclusive_scan(ex, f, l, out_it, init, binary_combine_op);
```

- **Combine operation** defaulted to `std::plus<>`:

```
| exclusive_scan(f, l, out_it, init);  
| exclusive_scan(ex, f, l, out_it, init);
```

- **No option** for defaulting **initial value**.

Parallel Algorithms

J. Daniel García

Universidad Carlos III de Madrid

